

# Module 7 - Persister les donnees avec MongoDB et Mongoose

*Support de cours - Projet fil rouge Express / EJS / CRUD produits*

## Sommaire

- 1. Le probleme actuel de notre application
- 2. Pourquoi utiliser une base de donnees ?
- 3. SQL et NoSQL : comprendre simplement
- 4. C'est quoi le NoSQL ?
- 5. Comparaison SQL / MongoDB
- 6. C'est quoi MongoDB ?
- 7. Les notions importantes dans MongoDB
- 8. Comprendre les relations en NoSQL
- 9. Premiere methode : integrer les donnees
- 10. Deuxieme methode : utiliser des references
- 11. Relations simples : comparaison avec SQL
- 12. Installer MongoDB sur Windows
- 13. Installer ou verifier MongoDB Compass
- 14. Utiliser MongoDB Compass
- 15. Installer Mongoose dans le projet
- 16. Creer le fichier de connexion a MongoDB
- 17. Connecter la base dans app.js
- 18. Creer le modele Product
- 19. Supprimer l'ancien fichier data/products.js ?
- 20. Modifier le controleur des produits
- 21. Version complete de controllers/productController.js
- 22. Explication du controleur
- 23. Modifier routes/productRoutes.js
- 24. Modifier views/products.ejs
- 25. Modifier views/product-details.ejs
- 26. Modifier views/product-form.ejs
- 27. Modifier la page d'accueil
- 28. Verifier app.js complet
- 29. Structure finale du projet
- 30. Tester l'application
- 31. Comprendre la persistance
- 32. Validation simple avec Mongoose
- 33. CRUD avec MongoDB : resume
- 34. Liaison entre modele, controleur et vue
- 35. Ce que Compass permet de montrer aux participants
- 36. Erreurs frequentes
- 37. Petit exercice guide
- 38. Synthese pedagogique
- 39. Conclusion du module

## Objectif general du module

Jusqu'a maintenant, notre application Express permet de gerer des produits avec une interface EJS.

Nous avons deja une application avec :

- une page d'accueil ;
- une liste de produits ;
- un detail produit ;
- un formulaire d'ajout ;
- un formulaire de modification ;
- une suppression ;
- une recherche ;
- des routes ;
- des controleurs ;
- des vues EJS ;
- un debut d'organisation MVC.

Mais il reste un probleme important.

Les produits sont encore stockes dans un tableau JavaScript.

Cela veut dire que si on redemarre le serveur, les produits ajoutes disparaissent.

Dans ce module, nous allons remplacer le tableau JavaScript par une vraie base de donnees MongoDB.

A la fin du module, l'application permettra de :

- comprendre la difference entre SQL et NoSQL ;
- comprendre MongoDB ;
- comprendre les notions de base : base de donnees, collection, document, champ ;
- installer MongoDB sur Windows ;
- utiliser MongoDB Compass ;
- connecter Express a MongoDB ;
- utiliser Mongoose ;
- creer un modele Product ;
- definir un schema ;
- ajouter une validation simple ;
- lire les produits depuis MongoDB ;
- ajouter un produit dans MongoDB ;
- afficher le detail d'un produit depuis MongoDB ;
- modifier un produit dans MongoDB ;
- supprimer un produit dans MongoDB ;
- afficher les donnees persistees dans les vues EJS.

## 1. Le probleme actuel de notre application

### Situation actuelle

Dans les modules precedents, nous avons utilise un tableau JavaScript pour stocker les produits.

Exemple :

```
export const products = [  
  {  
    id: 1,  
    name: "Clavier mecanique",  
    price: 8500,  
    stock: 12,
```

```

    category: "Accessoires",
    description: "Un clavier confortable pour ecrire du code."
  },
  {
    id: 2,
    name: "Souris sans fil",
    price: 3200,
    stock: 4,
    category: "Accessoires",
    description: "Une souris simple et pratique pour le travail quotidien."
  }
];

```

Ce tableau est pratique pour apprendre.

Mais il a une limite importante.

Quand le serveur redemarre, les donnees ajoutees pendant l'execution disparaissent.

## Pourquoi ?

Parce qu'un tableau JavaScript existe seulement en memoire.

L'image simple a retenir :

Un tableau JavaScript, c'est comme ecrire sur un tableau blanc.

Tant que le tableau est la, on voit les donnees.

Mais des qu'on efface ou qu'on redemarre, tout disparaît.

Une base de donnees, c'est comme ecrire dans un cahier.

Meme si on ferme l'application, les donnees restent.

## 2. Pourquoi utiliser une base de donnees ?

Une base de donnees permet de conserver les informations de maniere durable.

Dans notre application, on veut que les produits restent enregistres meme si :

- le serveur redemarre ;
- l'ordinateur redemarre ;
- l'application est arretee puis relancee ;
- plusieurs utilisateurs ajoutent des donnees ;
- le nombre de produits augmente.

Une base de donnees permet aussi de faire des operations importantes :

- creer des donnees ;
- lire des donnees ;
- modifier des donnees ;
- supprimer des donnees ;
- rechercher des donnees ;
- trier des donnees ;
- filtrer des donnees.

Ces operations sont souvent appelees CRUD.

CRUD signifie :

- Create : creer ;
- Read : lire ;
- Update : modifier ;
- Delete : supprimer.

Dans notre application produits, cela correspond a :

- ajouter un produit ;
- afficher les produits ;
- afficher le detail d'un produit ;
- modifier un produit ;
- supprimer un produit.

### 3. SQL et NoSQL : comprendre simplement

#### Les bases SQL

Les bases SQL sont les bases de donnees relationnelles classiques.

Exemples :

- MySQL ;
- PostgreSQL ;
- SQL Server ;
- Oracle.

Dans une base SQL, les donnees sont souvent organisees sous forme de tables.

Exemple d'une table products :

| id | name              | price | stock | category    |
|----|-------------------|-------|-------|-------------|
| 1  | Clavier mecanique | 8500  | 12    | Accessoires |
| 2  | Souris sans fil   | 3200  | 4     | Accessoires |

On peut imaginer une table comme un tableau Excel.

Chaque ligne represente un element.

Chaque colonne represente une information.

#### Exemple SQL

```
SELECT * FROM products;
```

Cette requete veut dire :

Donne-moi tous les produits.

Autre exemple :

```
SELECT * FROM products WHERE price > 5000;
```

Cela veut dire :

Donne-moi les produits dont le prix est superieur a 5000.

### 4. C'est quoi le NoSQL ?

NoSQL signifie generalement : base de donnees non relationnelle.

Cela ne veut pas dire que SQL est mauvais.

Cela veut simplement dire que la base ne fonctionne pas exactement comme une base relationnelle classique.

MongoDB est une base NoSQL orientee documents.

Au lieu de stocker les données dans des lignes et colonnes, MongoDB stocke les données sous forme de documents.

Un document ressemble beaucoup à un objet JavaScript.

Exemple :

```
{
  name: "Clavier mecanique",
  price: 8500,
  stock: 12,
  category: "Accessoires",
  description: "Un clavier confortable pour ecrire du code."
}
```

Pour un développeur JavaScript, MongoDB est assez naturel, car les documents ressemblent à des objets JavaScript.

MongoDB utilise un modèle flexible, et Mongoose permet d'ajouter une structure plus stricte avec des schémas et des modèles.

## 5. Comparaison SQL / MongoDB

### En SQL

On parle souvent de :

- base de données ;
- table ;
- ligne ;
- colonne ;
- cle primaire ;
- cle étrangere.

### En MongoDB

On parle plutôt de :

- database ;
- collection ;
- document ;
- field ;
- `\_id` ;
- ObjectId.

### Tableau de comparaison

| SQL             | MongoDB                                |
|-----------------|----------------------------------------|
| Base de données | Base de données                        |
| Table           | Collection                             |
| Ligne           | Document                               |
| Colonne         | Champ                                  |
| id              | `_id`                                  |
| Cle étrangere   | Reference ObjectId                     |
| JOIN            | populate ou organisation des documents |

## Exemple simple

En SQL, on aurait une table :

```
products
```

Dans MongoDB, on aura une collection :

```
products
```

Dans cette collection, chaque produit sera un document.

Exemple de document produit :

```
{
  _id: ObjectId("665f2b9e9d3a2c0012a45678"),
  name: "Clavier mecanique",
  price: 8500,
  stock: 12,
  category: "Accessoires",
  description: "Un clavier confortable pour ecrire du code."
}
```

## Point important

Dans MongoDB, chaque document recoit automatiquement un identifiant appele :

```
_id
```

Avant, dans notre tableau JavaScript, on avait :

```
id: 1
```

Avec MongoDB, on utilisera :

```
_id
```

Donc dans nos liens EJS, on passera progressivement de :

```
/products/<%= product.id %>
```

a :

```
/products/<%= product._id %>
```

## 6. C'est quoi MongoDB ?

MongoDB est une base de donnees NoSQL orientee documents.

Elle permet de stocker des donnees sous une forme proche des objets JavaScript.

Dans notre cas, elle va stocker les produits.

Exemple :

```
{
  name: "Ecran 24 pouces",
  price: 28000,
  stock: 8,
  category: "Ecrans",
  description: "Un ecran adapte au developpement web et a la bureautique."
}
```

Au lieu d'avoir un fichier `data/products.js`, nous aurons une vraie collection MongoDB.

## 7. Les notions importantes dans MongoDB

### 7.1 Database

Une database est une base de données.

Dans notre projet, on peut créer une base appelée :

```
gestion_produits_ejs
```

Cette base contiendra les données de notre application.

### 7.2 Collection

Une collection est un groupe de documents.

Exemple :

```
products
```

La collection products contiendra tous les produits.

### 7.3 Document

Un document est un élément stocké dans une collection.

Exemple :

```
{
  name: "Souris sans fil",
  price: 3200,
  stock: 4,
  category: "Accessoires",
  description: "Une souris simple et pratique pour le travail quotidien."
}
```

### 7.4 Field

Un field est un champ du document.

Exemples :

```
name
price
stock
category
description
```

### 7.5 \_id

Chaque document MongoDB possède un identifiant unique.

Ce champ s'appelle :

```
_id
```

Il permet de retrouver un document précis.

## 8. Comprendre les relations en NoSQL

### Le réflexe SQL

Quand on vient du SQL, on pense souvent avec des relations.

Exemple :

Un produit appartient à une catégorie.

En SQL, on pourrait avoir deux tables :

```
products
categories
```

Et dans la table products, on aurait :

```
category_id
```

C'est une cle étrangere.

## En MongoDB, on a deux grandes possibilites

Avec MongoDB, on peut faire autrement.

On peut :

- integrer les donnees directement dans le document ;
- ou utiliser des references entre documents.

## 9. Premiere methode : integrer les donnees

### Exemple

Au lieu d'avoir une collection separee pour les categories, on peut ecrire directement :

```
{
  name: "Clavier mecanique",
  price: 8500,
  stock: 12,
  category: "Accessoires",
  description: "Un clavier confortable pour ecrire du code."
}
```

Ici, la categorie est juste un champ texte.

C'est simple.

Pour notre niveau actuel, cette solution est tres bien.

### Quand utiliser cette methode ?

On peut integrer les donnees quand :

- l'information est simple ;
- l'information ne change pas souvent ;
- on n'a pas besoin d'une vraie gestion independante ;
- on veut garder le projet facile a comprendre.

Dans notre application, garder :

```
category: "Accessoires"
```

est suffisant pour commencer.

## 10. Deuxieme methode : utiliser des references

### Exemple

Imaginons maintenant qu'on veuille gerer les categories separement.

On aurait une collection categories :

```
{
  _id: ObjectId("111"),
  name: "Accessoires"
}
```



}

Et une collection products :

```
{
  name: "Clavier mecanique",
  price: 8500,
  category: ObjectId("111")
}
```

Ici, le produit ne stocke pas directement le nom de la categorie.

Il stocke l'identifiant de la categorie.

C'est une reference.

## Avec Mongoose

Avec Mongoose, on pourrait ecrire :

```
category: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "Category"
}
```

Cela veut dire :

Ce produit est lie a un document de la collection des categories.

## A retenir pour ce module

Pour rester niveau 1, nous allons garder la categorie comme un simple texte.

Donc notre produit aura :

```
category: String
```

Plus tard, quand le niveau sera plus avance, on pourra creer un modele Category.

# 11. Relations simples : comparaison avec SQL

## Relation 1 - 1

Exemple :

Un utilisateur a un profil.

En SQL :

```
users
profiles
```

En MongoDB, on peut integrer le profil dans l'utilisateur :

```
{
  name: "Ali",
  profile: {
    age: 25,
    city: "Alger"
  }
}
```

## Relation 1 - N

Exemple :

Une categorie contient plusieurs produits.

Solution simple niveau 1 :

```
{
```

```
name: "Clavier mecanique",
category: "Accessoires"
}
```

Solution plus avancée :

```
{
  name: "Clavier mecanique",
  category: ObjectId("id_de_la_categorie")
}
```

## Relation N - N

Exemple :

Un étudiant peut suivre plusieurs cours.

Un cours peut contenir plusieurs étudiants.

En MongoDB, on peut utiliser un tableau d'identifiants :

```
{
  name: "Ahmed",
  courses: [
    ObjectId("id_cours_1"),
    ObjectId("id_cours_2")
  ]
}
```

## Règle simple pour débiter

Si la donnée est petite et appartient clairement au document, on peut l'intégrer.

Si la donnée est grande, réutilisée, ou doit être gérée séparément, on peut utiliser une référence.

# 12. Installer MongoDB sur Windows

## Objectif

Installer MongoDB localement sur les machines Windows des participants.

Nous allons utiliser :

- MongoDB Community Server ;
- MongoDB Compass.

MongoDB Community Edition peut être installé sur Windows avec un installateur .msi.

## Étape 1 - Télécharger MongoDB Community Server

Aller sur la page officielle de téléchargement de MongoDB Community Server.

Choisir :

```
Version : MongoDB Community Server
Platform : Windows
Package : MSI
```

Puis cliquer sur Download.

## Étape 2 - Lancer l'installation

Une fois le fichier .msi téléchargé :

1. Aller dans le dossier Téléchargements.
2. Double-cliquer sur le fichier .msi.
3. Lancer l'assistant d'installation.

### Etape 3 - Choisir le type d'installation

Choisir :

`Complete`

C'est le plus simple pour une formation.

L'option Complete installe MongoDB et les outils dans l'emplacement par défaut.

### Etape 4 - Installer MongoDB comme service Windows

Pendant l'installation, garder l'option :

`Install MongoDB as a Service`

Cela permet à MongoDB de démarrer comme un service Windows.

C'est plus simple pour les participants.

### Etape 5 - Installer MongoDB Compass

Pendant l'installation, garder aussi l'option :

`Install MongoDB Compass`

MongoDB Compass est l'interface graphique qui permet de voir les bases, les collections et les documents.

### Etape 6 - Verifier que MongoDB fonctionne

Après installation :

4. Ouvrir le menu Windows.
5. Chercher `Services`.
6. Chercher le service `MongoDB`.
7. Verifier qu'il est démarré.

Si le service n'est pas démarré :

- clic droit sur MongoDB ;
- Start / Démarrer.

## 13. Installer ou vérifier MongoDB Compass

Normalement, Compass peut être installé pendant l'installation de MongoDB.

Si Compass n'est pas installé, on peut l'installer séparément depuis la page officielle MongoDB Compass.

### Ouvrir Compass

Après installation :

8. Ouvrir MongoDB Compass.
9. Dans le champ de connexion, utiliser :

`mongodb://127.0.0.1:27017`

10. Cliquer sur Connect.

Si tout est correct, Compass affiche les bases de données locales.

## 14. Utiliser MongoDB Compass

### Objectif

Comprendre visuellement MongoDB avant d'ecrire du code.

Compass permet de voir :

- les databases ;
- les collections ;
- les documents ;
- les champs ;
- les valeurs.

### Creer une base de donnees

Dans Compass :

11. Cliquer sur Create Database.

12. Database Name :

```
gestion_produits_ejs
```

13. Collection Name :

```
products
```

14. Cliquer sur Create Database.

### Ajouter un document a la main

Dans la collection products, cliquer sur Insert Document.

Ajouter par exemple :

```
{
  "name": "Clavier mecanique",
  "price": 8500,
  "stock": 12,
  "category": "Accessoires",
  "description": "Un clavier confortable pour ecrire du code."
}
```

Puis valider.

MongoDB ajoutera automatiquement un champ `_id`.

### A retenir

Compass est tres utile pour verifier ce que fait notre application.

Quand on ajoutera un produit depuis le formulaire EJS, on pourra voir le produit apparaitre dans Compass.

## 15. Installer Mongoose dans le projet

### Objectif

Permettre a notre application Express de communiquer avec MongoDB.

Dans le terminal, a la racine du projet :

```
npm install mongoose
```

Mongoose est un ODM pour MongoDB : il permet de definir des schemas, des modeles, de valider les donnees et de manipuler les documents.

## Pourquoi utiliser Mongoose ?

Express seul ne sait pas parler directement a MongoDB.

Mongoose va servir d'intermediaire entre :

- notre application Express ;
- notre base MongoDB.

On peut le voir comme un traducteur.

Express dit :

Je veux creer un produit.

Mongoose transforme cela en operation MongoDB.

MongoDB enregistre le document.

## 16. Creer le fichier de connexion a MongoDB

### Objectif

Centraliser la connexion a la base de donnees.

Creer un dossier :

```
config
```

Puis creer le fichier :

```
config/database.js
```

### Code de config/database.js

```
import mongoose from "mongoose";

export async function connectDB() {
  try {
    await mongoose.connect("mongodb://127.0.0.1:27017/gestion_produits_ejs");

    console.log("Connexion a MongoDB reussie");
  } catch (error) {
    console.error("Erreur de connexion a MongoDB");
    console.error(error.message);
    process.exit(1);
  }
}
```

### Explication

Cette ligne :

```
await mongoose.connect("mongodb://127.0.0.1:27017/gestion_produits_ejs");
```

signifie :

Connecte l'application a MongoDB local.

Detail de l'adresse :

```
mongodb://127.0.0.1:27017/gestion_produits_ejs
```

- `mongodb://` : protocole MongoDB ;
- `127.0.0.1` : machine locale ;
- `27017` : port par defaut de MongoDB ;
- `gestion\_produits\_ejs` : nom de la base de donnees.

Si cette base n'existe pas encore, MongoDB pourra la creer au moment ou on insere des donnees.

## 17. Connecter la base dans app.js

### Objectif

Faire la connexion MongoDB au démarrage de l'application.

Modifier `app.js`.

### app.js

```
import express from "express";

import { connectDB } from "../config/database.js";

import pageRoutes from "../routes/pageRoutes.js";
import productRoutes from "../routes/productRoutes.js";

const app = express();
const PORT = 3000;

app.set("view engine", "ejs");

app.use(express.static("public"));
app.use(express.urlencoded({ extended: true }));

app.use("/", pageRoutes);
app.use("/products", productRoutes);

app.use((req, res) => {
  res.status(404).render("404", {
    title: "Page introuvable"
  });
});

async function startServer() {
  await connectDB();

  app.listen(PORT, () => {
    console.log(`Serveur lance sur http://localhost:${PORT}`);
  });
}

startServer();
```

### Explication

Avant, on lançait directement le serveur.

Maintenant, on fait d'abord :

```
await connectDB();
```

Puis seulement après, on lance le serveur.

Cela permet de vérifier que la base de données est disponible.

## 18. Créer le modèle Product

### Objectif

Remplacer le fichier `data/products.js` par un modèle Mongoose.

Créer un dossier :

`models`

Puis créer :

`models/Product.js`

## Code de models/Product.js

```
import mongoose from "mongoose";

const productSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, "Le nom du produit est obligatoire."],
      trim: true
    },

    price: {
      type: Number,
      required: [true, "Le prix est obligatoire."],
      min: [0, "Le prix ne peut pas etre negatif."]
    },

    stock: {
      type: Number,
      required: [true, "Le stock est obligatoire."],
      min: [0, "Le stock ne peut pas etre negatif."],
      default: 0
    },

    category: {
      type: String,
      default: "Non classe",
      trim: true
    },

    description: {
      type: String,
      required: [true, "La description est obligatoire."],
      trim: true
    }
  },
  {
    timestamps: true
  }
);

const Product = mongoose.model("Product", productSchema);

export default Product;
```

## Explication

Cette partie :

```
const productSchema = new mongoose.Schema(...)
```

definit la structure d'un produit.

## Exemple de champ

```
name: {
  type: String,
  required: [true, "Le nom du produit est obligatoire."],
  trim: true
}
```

Cela veut dire :

- le nom est une chaine de caracteres ;
- le nom est obligatoire ;

- les espaces inutiles au debut et a la fin seront supprimes.

## timestamps

Cette option :

```
{
  timestamps: true
}
```

ajoute automatiquement deux champs :

```
createdAt
updatedAt
```

Cela permet de savoir quand un produit a ete cree ou modifie.

## Product devient products

Cette ligne :

```
const Product = mongoose.model("Product", productSchema);
```

creee le modele Product.

Mongoose utilise generalement le nom singulier du modele et le transforme en nom de collection au pluriel et en minuscules.

Donc ici :

```
Product
```

sera associe a la collection :

```
products
```

## 19. Supprimer l'ancien fichier data/products.js ?

### Objectif

Comprendre ce qui change.

Avant, on avait :

```
data/products.js
```

Maintenant, les produits sont dans MongoDB.

Donc le fichier data/products.js ne sera plus utilise par le controleur des produits.

On peut le garder temporairement comme souvenir ou le supprimer.

Dans le vrai projet, on n'en a plus besoin.

## 20. Modifier le controleur des produits

### Objectif

Remplacer les operations sur le tableau par des operations MongoDB.

Avant, on faisait :

```
products.push(newProduct);
```

Maintenant, on fera :

```
await Product.create(newProduct);
```

Avant, on faisait :



```
products.find(...)
```

Maintenant, on fera :

```
await Product.findById(...)
```

Avant, on faisait :

```
products.filter(...)
```

Maintenant, on fera une recherche MongoDB.

## 21. Version complete de controllers/productController.js

Remplacer le contenu de :

```
controllers/productController.js
```

par :

```
import mongoose from "mongoose";
import Product from "../models/Product.js";

export async function getProducts(req, res) {
  const search = req.query.search || "";
  const message = req.query.message || "";

  const filter = {};

  if (search !== "") {
    filter.name = {
      $regex: search,
      $options: "i"
    };
  }

  const products = await Product.find(filter).sort({ createdAt: -1 });

  res.render("products", {
    title: "Liste des produits",
    products: products,
    search: search,
    message: message
  });
}

export async function getProductDetails(req, res) {
  const productId = req.params.id;

  if (!mongoose.Types.ObjectId.isValid(productId)) {
    return res.status(404).render("404", {
      title: "Produit introuvable"
    });
  }

  const product = await Product.findById(productId);

  if (!product) {
    return res.status(404).render("404", {
      title: "Produit introuvable"
    });
  }

  res.render("product-details", {
    title: "Detail du produit",
    product: product
  });
}

export function showCreateProductForm(req, res) {
  res.render("product-form", {
    title: "Ajouter un produit",
```

```

    action: "/products",
    buttonLabel: "Ajouter",
    error: "",
    oldInput: {
      name: "",
      price: "",
      stock: "",
      category: "",
      description: ""
    }
  });
}

export async function createProduct(req, res) {
  try {
    await Product.create({
      name: req.body.name,
      price: req.body.price,
      stock: req.body.stock,
      category: req.body.category,
      description: req.body.description
    });

    res.redirect("/products?message=created");
  } catch (error) {
    res.status(400).render("product-form", {
      title: "Ajouter un produit",
      action: "/products",
      buttonLabel: "Ajouter",
      error: getValidationMessage(error),
      oldInput: {
        name: req.body.name,
        price: req.body.price,
        stock: req.body.stock,
        category: req.body.category,
        description: req.body.description
      }
    });
  }
}

export async function showEditProductForm(req, res) {
  const productId = req.params.id;

  if (!mongoose.Types.ObjectId.isValid(productId)) {
    return res.status(404).render("404", {
      title: "Produit introuvable"
    });
  }

  const product = await Product.findById(productId);

  if (!product) {
    return res.status(404).render("404", {
      title: "Produit introuvable"
    });
  }

  res.render("product-form", {
    title: "Modifier un produit",
    action: `/products/${product._id}/update`,
    buttonLabel: "Modifier",
    error: "",
    oldInput: {
      name: product.name,
      price: product.price,
      stock: product.stock,
      category: product.category,
      description: product.description
    }
  });
}

```

```

export async function updateProduct(req, res) {
  const productId = req.params.id;

  if (!mongoose.Types.ObjectId.isValid(productId)) {
    return res.status(404).render("404", {
      title: "Produit introuvable"
    });
  }

  try {
    const product = await Product.findByIdAndUpdate(
      productId,
      {
        name: req.body.name,
        price: req.body.price,
        stock: req.body.stock,
        category: req.body.category,
        description: req.body.description
      },
      {
        new: true,
        runValidators: true
      }
    );

    if (!product) {
      return res.status(404).render("404", {
        title: "Produit introuvable"
      });
    }

    res.redirect("/products?message=updated");
  } catch (error) {
    res.status(400).render("product-form", {
      title: "Modifier un produit",
      action: `/products/${productId}/update`,
      buttonLabel: "Modifier",
      error: getValidationMessage(error),
      oldInput: {
        name: req.body.name,
        price: req.body.price,
        stock: req.body.stock,
        category: req.body.category,
        description: req.body.description
      }
    });
  }
}

export async function deleteProduct(req, res) {
  const productId = req.params.id;

  if (!mongoose.Types.ObjectId.isValid(productId)) {
    return res.status(404).render("404", {
      title: "Produit introuvable"
    });
  }

  await Product.findByIdAndDelete(productId);

  res.redirect("/products?message=deleted");
}

function getValidationMessage(error) {
  if (error.name === "ValidationError") {
    const messages = Object.values(error.errors).map((item) => {
      return item.message;
    });

    return messages.join(" ");
  }
}

```

```
return "Une erreur est survenue. Veuillez verifier les informations saisies.";
}
```

## 22. Explication du controleur

### Importer le modele

```
import Product from "../models/Product.js";
```

On n'importe plus le tableau :

```
import { products } from "../data/products.js";
```

Maintenant, on importe le modele Mongoose.

### Lire les produits

```
const products = await Product.find(filter).sort({ createdAt: -1 });
```

Cela veut dire :

Cherche les produits dans MongoDB.

Puis trie les produits du plus recent au plus ancien.

### Faire une recherche

```
filter.name = {
  $regex: search,
  $options: "i"
};
```

Cela veut dire :

Cherche les produits dont le nom contient le texte recherche.

L'option "i" veut dire :

Ne pas tenir compte des majuscules et minuscules.

Donc si on cherche :

```
clavier
```

on peut trouver :

```
Clavier mecanique
```

### Ajouter un produit

```
await Product.create({
  name: req.body.name,
  price: req.body.price,
  stock: req.body.stock,
  category: req.body.category,
  description: req.body.description
});
```

Cela cree un nouveau document dans MongoDB.

### Trouver un produit par son id

```
const product = await Product.findById(productId);
```

Cela cherche un produit dans MongoDB avec son `_id`.

### Modifier un produit

```
await Product.findByIdAndUpdate(...)
```

Cela modifie un document existant.

L'option :

```
runValidators: true
```

demande a Mongoose d'appliquer les validations du schema pendant la modification.

## Supprimer un produit

```
await Product.findByIdAndDelete(productId);
```

Cela supprime le produit dans MongoDB.

## 23. Modifier routes/productRoutes.js

### Objectif

Ajouter les routes CRUD completes avec MongoDB.

Remplacer le contenu de :

```
routes/productRoutes.js
```

par :

```
import express from "express";

import {
  getProducts,
  getProductDetails,
  showCreateProductForm,
  createProduct,
  showEditProductForm,
  updateProduct,
  deleteProduct
} from "../controllers/productController.js";

const router = express.Router();

router.get("/", getProducts);

router.get("/new", showCreateProductForm);
router.post("/", createProduct);

router.get("/:id/edit", showEditProductForm);
router.post("/:id/update", updateProduct);
router.post("/:id/delete", deleteProduct);

router.get("/:id", getProductDetails);

export default router;
```

### Explication importante sur l'ordre

Cette route :

```
router.get("/new", showCreateProductForm);
```

doit etre avant :

```
router.get("/:id", getProductDetails);
```

Sinon Express pourrait comprendre :

```
/products/new
```

comme :

```
/products/:id
```

avec :

```
id = new
```

Meme chose pour :

```
router.get("/:id/edit", showEditProductForm);
```

Elle doit être placée avant la route détail :

```
router.get("/:id", getProductDetails);
```

## 24. Modifier views/products.ejs

### Objectif

Afficher les produits venant de MongoDB.

Attention : on utilise maintenant :

```
product._id
```

et non plus :

```
product.id
```

### Code de views/products.ejs

```
<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= title %></h2>

  <% if (message === "created") { %>
    <p class="success">Produit ajouté avec succès.</p>
  <% } %>

  <% if (message === "updated") { %>
    <p class="success">Produit modifié avec succès.</p>
  <% } %>

  <% if (message === "deleted") { %>
    <p class="success">Produit supprimé avec succès.</p>
  <% } %>

  <form action="/products" method="GET">
    <input
      type="text"
      name="search"
      placeholder="Rechercher un produit"
      value="<%= search %>"
    >
    <button type="submit">Rechercher</button>
  </form>

  <% if (search) { %>
    <p>
      Recherche actuelle :
      <strong><%= search %></strong>
    </p>

    <a href="/products">Reinitialiser</a>
  <% } %>

  <p>
    <a href="/products/new">Ajouter un produit</a>
  </p>

  <% if (products.length === 0) { %>
    <p>Aucun produit disponible.</p>
  <% } else { %>
    <ul>
      <% products.forEach((product) => { %>
        <li>
          <strong><%= product.name %></strong>
```

```

-
<%= product.price %> DA
-
Stock : <%= product.stock %>

<br>

<a href="/products/<%= product._id %>">Voir le detail</a>
|
<a href="/products/<%= product._id %>/edit">Modifier</a>

<form
  action="/products/<%= product._id %>/delete"
  method="POST"
  style="display:inline;"
>
  <button type="submit">Supprimer</button>
</form>
</li>
<% }) %>
</ul>
<% } %>
</main>

<%- include("partials/footer") %>

```

## Explication

Avant :

```
/products/<%= product.id %>
```

Maintenant :

```
/products/<%= product._id %>
```

Parce que l'identifiant vient de MongoDB.

## 25. Modifier views/product-details.ejs

### Objectif

Afficher le detail d'un produit venant de MongoDB.

### Code de views/product-details.ejs

```

<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= product.name %></h2>

  <p>Prix : <%= product.price %> DA</p>

  <p>Stock : <%= product.stock %></p>

  <% if (product.stock < 5) { %>
    <p class="error">Stock faible.</p>
  <% } %>

  <p>Categorie : <%= product.category %></p>

  <p><%= product.description %></p>

  <p>
    <a href="/products/<%= product._id %>/edit">Modifier</a>
  </p>

  <form action="/products/<%= product._id %>/delete" method="POST">

```

```

    <button type="submit">Supprimer</button>
  </form>

  <p>
    <a href="/products">Retour a la liste</a>
  </p>
</main>

<%- include("partials/footer") %>

```

## 26. Modifier views/product-form.ejs

### Objectif

Utiliser un seul formulaire pour :

- ajouter un produit ;
- modifier un produit.

Pour cela, on utilise des variables envoyées par le contrôleur :

```

action
buttonLabel
oldInput
error

```

### Code de views/product-form.ejs

```

<%- include("partials/header") %>
<%- include("partials/navbar") %>

<main>
  <h2><%= title %></h2>

  <% if (error) { %>
    <p class="error"><%= error %></p>
  <% } %>

  <form action="<%= action %>" method="POST">
    <div>
      <label for="name">Nom du produit</label>
      <input
        type="text"
        id="name"
        name="name"
        value="<%= oldInput.name %>"
      >
    </div>

    <div>
      <label for="price">Prix</label>
      <input
        type="number"
        id="price"
        name="price"
        value="<%= oldInput.price %>"
      >
    </div>

    <div>
      <label for="stock">Stock</label>
      <input
        type="number"
        id="stock"
        name="stock"
        value="<%= oldInput.stock %>"
      >
    </div>
  </form>

```



```

<div>
  <label for="category">Categorie</label>
  <input
    type="text"
    id="category"
    name="category"
    value="<%= oldInput.category %>"
  >
</div>

<div>
  <label for="description">Description</label>
  <textarea
    id="description"
    name="description"
  ><%= oldInput.description %></textarea>
</div>

  <button type="submit"><%= buttonLabel %></button>
</form>

<p>
  <a href="/products">Retour a la liste</a>
</p>
</main>

<%- include("partials/footer") %>

```

## Explication

Pour l'ajout, le controleur envoie :

```

action: "/products",
buttonLabel: "Ajouter"

```

Pour la modification, le controleur envoie :

```

action: `/products/${product._id}/update`,
buttonLabel: "Modifier"

```

Donc le meme fichier EJS peut servir pour deux actions differentes.

## 27. Modifier la page d'accueil

### Objectif

Afficher le nombre de produits venant de MongoDB.

Avant, dans `pageController.js`, on utilisait :

```
products.length
```

Maintenant, on doit demander a MongoDB de compter les documents.

### Code de controllers/pageController.js

```

import Product from "../models/Product.js";

export async function getHomePage(req, res) {
  const productCount = await Product.countDocuments();

  res.render("home", {
    title: "Accueil",
    productCount: productCount
  });
}

```

## Explication

Cette ligne :

```
const productCount = await Product.countDocuments();
```

demande a MongoDB :

Combien de documents existent dans la collection des produits ?

## 28. Verifier app.js complet

Le fichier app.js complet doit ressembler a ceci :

```
import express from "express";

import { connectDB } from "../config/database.js";

import pageRoutes from "../routes/pageRoutes.js";
import productRoutes from "../routes/productRoutes.js";

const app = express();
const PORT = 3000;

app.set("view engine", "ejs");

app.use(express.static("public"));
app.use(express.urlencoded({ extended: true }));

app.use("/", pageRoutes);
app.use("/products", productRoutes);

app.use((req, res) => {
  res.status(404).render("404", {
    title: "Page introuvable"
  });
});

async function startServer() {
  await connectDB();

  app.listen(PORT, () => {
    console.log(`Serveur lance sur http://localhost:${PORT}`);
  });
}

startServer();
```

## 29. Structure finale du projet

A la fin de ce module, la structure ressemble a ceci :

```
gestion-produits-ejs/
|-- app.js
|-- package.json
|-- config/
|   |-- database.js
|-- models/
|   |-- Product.js
|-- controllers/
|   |-- pageController.js
|   |-- productController.js
```

```

|-- routes/
|  |-- pageRoutes.js
|  |-- productRoutes.js
|-- views/
|  |-- home.ejs
|  |-- products.ejs
|  |-- product-details.ejs
|  |-- product-form.ejs
|  |-- 404.ejs
|  |-- partials/
|     |-- header.ejs
|     |-- navbar.ejs
|     |-- footer.ejs
|-- public/
|  |-- css/
|     |-- style.css

```

Le dossier data/ n'est plus necessaire.

## 30. Tester l'application

### Etape 1 - Verifier MongoDB

Avant de lancer le projet, verifier que MongoDB est demarre.

Sur Windows :

15. Ouvrir Services.
16. Chercher MongoDB.
17. Verifier que le service est demarre.

### Etape 2 - Lancer le projet

Dans le terminal :

```
npm run dev
```

On doit voir :

```

Connexion a MongoDB reussie
Serveur lance sur http://localhost:3000

```

### Etape 3 - Ouvrir l'application

Dans le navigateur :

```
http://localhost:3000
```

Puis aller vers :

```
http://localhost:3000/products
```

### Etape 4 - Ajouter un produit

Aller sur :

```
http://localhost:3000/products/new
```

Remplir :

```

Nom : Casque audio
Prix : 6500
Stock : 10
Categorie : Audio
Description : Casque confortable pour travailler.

```

Cliquer sur Ajouter.

Le produit doit apparaître dans la liste.

## Etape 5 - Verifier dans Compass

Ouvrir MongoDB Compass.

Se connecter a :

```
mongodb://127.0.0.1:27017
```

Ouvrir :

```
gestion_produits_ejs
```

Puis :

```
products
```

Le produit ajoute depuis l'application doit etre visible dans Compass.

## 31. Comprendre la persistance

### Avant MongoDB

Quand on ajoutait un produit, il etait ajoute dans un tableau JavaScript.

Si on redemarrant le serveur, le produit disparaissait.

### Maintenant

Quand on ajoute un produit, il est enregistre dans MongoDB.

Si on redemarre le serveur, le produit reste.

C'est cela, la persistance des donnees.

## 32. Validation simple avec Mongoose

### Objectif

Empêcher les mauvaises donnees d'entrer dans la base.

Dans le modele Product, nous avons ecrit :

```
name: {
  type: String,
  required: [true, "Le nom du produit est obligatoire."],
  trim: true
}
```

Cela veut dire :

Le nom est obligatoire.

Si l'utilisateur laisse le nom vide, Mongoose renvoie une erreur.

Autre exemple :

```
price: {
  type: Number,
  required: [true, "Le prix est obligatoire."],
  min: [0, "Le prix ne peut pas etre negatif."]
}
```

Cela veut dire :

- le prix est obligatoire ;

- le prix ne peut pas être négatif.

## Exemple d'erreur

Si l'utilisateur écrit :

```
Prix : -100
```

Mongoose refuse l'enregistrement.

Le formulaire affiche un message d'erreur.

## 33. CRUD avec MongoDB : resume

### Create

Créer un produit :

```
await Product.create({
  name: req.body.name,
  price: req.body.price,
  stock: req.body.stock,
  category: req.body.category,
  description: req.body.description
});
```

### Read

Lire tous les produits :

```
const products = await Product.find();
```

Lire un seul produit :

```
const product = await Product.findById(req.params.id);
```

### Update

Modifier un produit :

```
await Product.findByIdAndUpdate(
  req.params.id,
  {
    name: req.body.name,
    price: req.body.price,
    stock: req.body.stock,
    category: req.body.category,
    description: req.body.description
  },
  {
    new: true,
    runValidators: true
  }
);
```

### Delete

Supprimer un produit :

```
await Product.findByIdAndDelete(req.params.id);
```

## 34. Liaison entre modele, controleur et vue

### Avant

La vue affichait des donnees venant d'un tableau.

```
data/products.js
```

### Maintenant

La vue affiche des donnees venant de MongoDB.

Le chemin devient :

```
MongoDB
  ↓
Mongoose Model
  ↓
Controller
  ↓
EJS View
  ↓
Navigateur
```

### Exemple

Dans le controleur :

```
const products = await Product.find();
```

Puis :

```
res.render("products", {
  products: products
});
```

Dans la vue :

```
<% products.forEach((product) => { %>
  <%= product.name %>
<% }) %>
```

Le navigateur recoit du HTML final.

## 35. Ce que Compass permet de montrer aux participants

Pendant la formation, Compass est tres utile pour rendre MongoDB visible.

On peut montrer :

- la base `gestion\_produits\_ejs` ;
- la collection `products` ;
- les documents ajoutes depuis le formulaire ;
- le champ `\_id` ;
- les champs `createdAt` et `updatedAt` ;
- la modification d'un produit ;
- la suppression d'un produit.

Exemple pedagogique :

18. Ajouter un produit depuis l'application.
19. Ouvrir Compass.
20. Actualiser la collection.
21. Montrer que le produit existe vraiment dans MongoDB.

Cela aide a comprendre que les donnees ne sont plus dans le code.

Elles sont maintenant dans la base.

## 36. Erreurs frequentes

### Erreur 1 - MongoDB n'est pas demarre

Message possible :

```
ECONNREFUSED
```

Cela signifie souvent que l'application n'arrive pas a se connecter a MongoDB.

Solution :

- ouvrir Services ;
- demarrer MongoDB ;
- relancer le projet.

### Erreur 2 - Mauvaise adresse MongoDB

Verifier dans config/database.js :

```
mongodb://127.0.0.1:27017/gestion_produits_ejs
```

### Erreur 3 - Oublier await

Mauvais exemple :

```
const products = Product.find();
```

Bon exemple :

```
const products = await Product.find();
```

Comme MongoDB travaille de maniere asynchrone, on utilise await.

### Erreur 4 - Utiliser product.id au lieu de product.\_id

Avec MongoDB, l'identifiant reel s'appelle :

```
_id
```

Donc dans les liens EJS :

```
/products/<%= product._id %>
```

### Erreur 5 - Placer les routes dans le mauvais ordre

Mauvais ordre :

```
router.get("/:id", getProductDetails);
router.get("/:id/edit", showEditProductForm);
```

Bon ordre :

```
router.get("/:id/edit", showEditProductForm);
router.get("/:id", getProductDetails);
```

## 37. Petit exercice guide

### Exercice 1 - Verifier la persistance

22. Ajouter un produit depuis le formulaire.
23. Arrêter le serveur.
24. Relancer le serveur.

25. Verifier que le produit est encore dans la liste.

Resultat attendu :

Le produit est toujours present.

## Exercice 2 - Verifier Compass

26. Ajouter un produit depuis l'application.

27. Ouvrir Compass.

28. Ouvrir la collection `products`.

29. Verifier que le produit existe.

## Exercice 3 - Tester la validation

Essayer d'ajouter un produit avec :

```
Nom vide
Prix : -50
Stock : -3
Description vide
```

Resultat attendu :

Le formulaire doit afficher un message d'erreur.

## Exercice 4 - Tester la recherche

Ajouter plusieurs produits :

```
Clavier mecanique
Souris sans fil
Ecran 24 pouces
Casque audio
```

Chercher :

```
clavier
```

Resultat attendu :

Seul le produit contenant `clavier` apparait.

## Exercice 5 - Modifier un produit

30. Cliquer sur Modifier.

31. Changer le prix.

32. Valider.

33. Verifier dans Compass que le prix a change.

## Exercice 6 - Supprimer un produit

34. Cliquer sur Supprimer.

35. Verifier que le produit disparaît.

36. Verifier dans Compass qu'il n'existe plus.

# 38. Synthese pedagogique

Dans ce module, nous avons remplace les donnees temporaires par une vraie base MongoDB.

Avant :

```
Tableau JavaScript
```

Maintenant :

```
MongoDB
```



Avant :

```
products.push(newProduct);
```

Maintenant :

```
await Product.create(newProduct);
```

Avant :

```
products.find(...)
```

Maintenant :

```
await Product.findById(...)
```

Avant :

```
products.filter(...)
```

Maintenant :

```
await Product.find(...)
```

Nous avons vu :

- la difference entre SQL et NoSQL ;
- le fonctionnement general de MongoDB ;
- les bases : database, collection, document, field ;
- la notion de `\_id` ;
- les relations en NoSQL ;
- l'installation de MongoDB sur Windows ;
- l'utilisation de MongoDB Compass ;
- l'installation de Mongoose ;
- la connexion a MongoDB ;
- la creation d'un modele ;
- la creation d'un schema ;
- la validation simple ;
- les operations CRUD avec MongoDB ;
- la liaison entre modeles, controleurs et vues ;
- l'affichage des donnees persistees dans l'interface.

## 39. Conclusion du module

Notre application est maintenant beaucoup plus proche d'une vraie application web.

Elle ne depend plus d'un simple tableau JavaScript.

Les produits sont enregistres dans MongoDB.

Les donnees restent disponibles meme apres redemarrage du serveur.

Nous avons maintenant une application Express / EJS / MongoDB organisee autour de cette logique :

```
Route
  ↓
Controller
  ↓
Model Mongoose
  ↓
MongoDB
  ↓
Controller
  ↓
View EJS
  ↓
Navigateur
```

C'est une etape tres importante.

Le projet fil rouge devient maintenant une vraie application web CRUD avec persistance des données.

## Sources officielles utiles

- Documentation MongoDB - Installation Windows : <https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-windows/>
- MongoDB Compass : <https://www.mongodb.com/products/tools/compass>
- Documentation Mongoose : <https://mongoosejs.com/docs/>
- Documentation Express : <https://expressjs.com/>